COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	08/18/2026	**DATE SUBMITTED:**	
NAME:	Elijah I. Dizor	**INSTRUCTOR:**	Engr. Jimlord Quejado

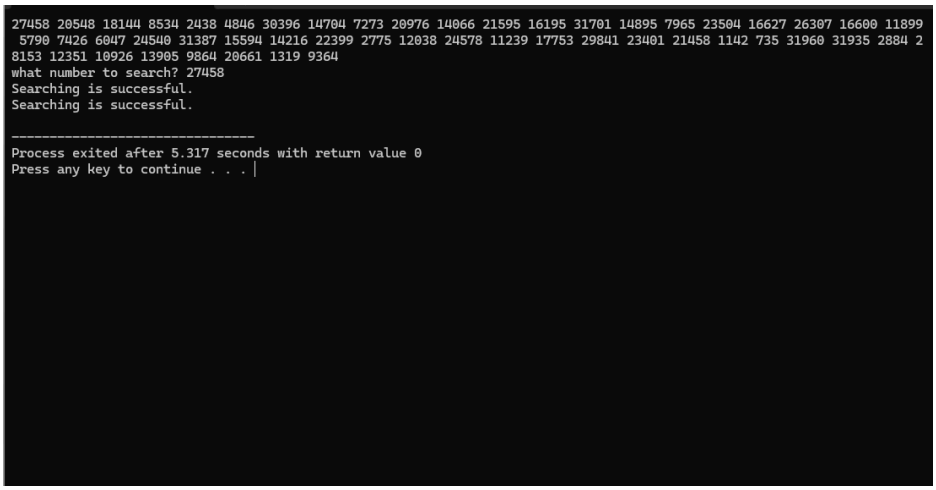
ACTIVITY NO. 6.1 Searching Techniques

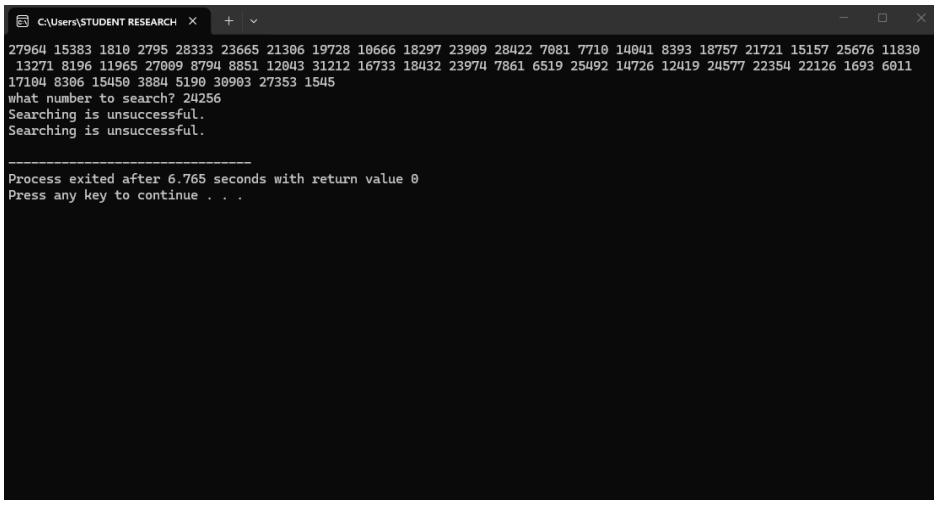
1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Create C++ code for searching data using different techniques

A.1 Sequential Search

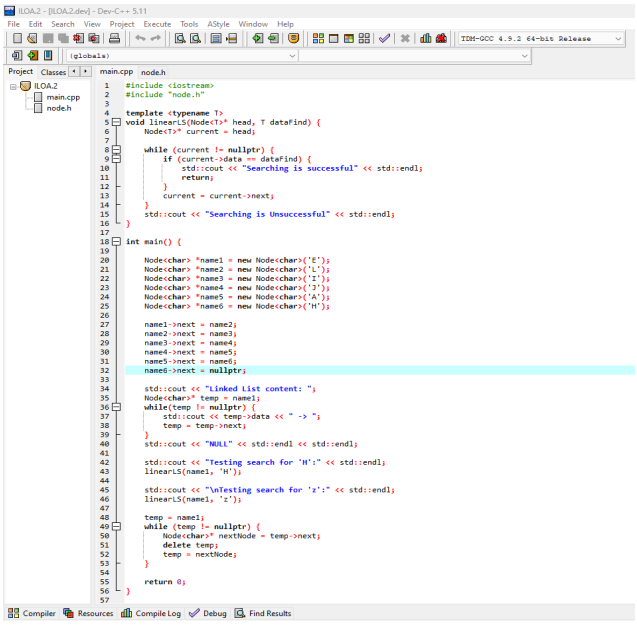
Screenshot	

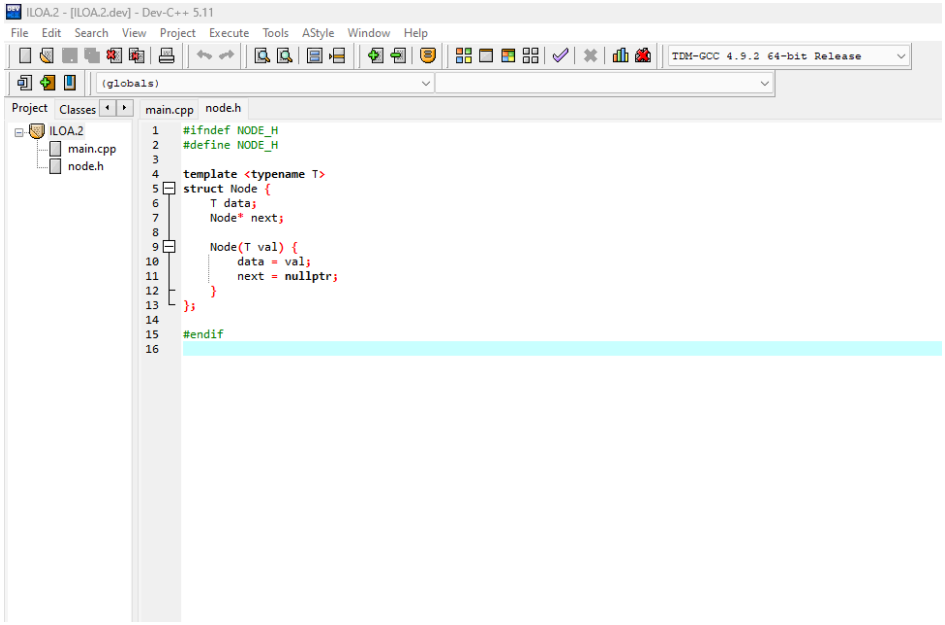


Observation	The linear search goes through an array element by element,the user input matches an item that was generated from rand() which creates random 4-5 digits. The conditions are based on whether that digit is on the list or not and prints out a message that 'searching was successful' , or 'searching was unsuccessful' if it does not match the digit that you are searching within the list.

Table 6-1. Data Generated and Observations.

Code





Output

```
31759 20423 17834 13262 20781 31773 726 26107 27854 17667 15882 14068 9574 32493 12917 2934 18897 1559 12532 63 5607 124
89 22849 3695 21841 14042 29544 20276 10395 12915 32122 8870 8329 7701 16772 15856 9135 29661 1986 25777 21052 20999 146
09 20898 23575 18610 1443 4728 32045 13538
what number to search? 31759
Searching is successful

--------------------------------
Process exited after 7.284 seconds with return value 0
Press any key to continue . . .
```

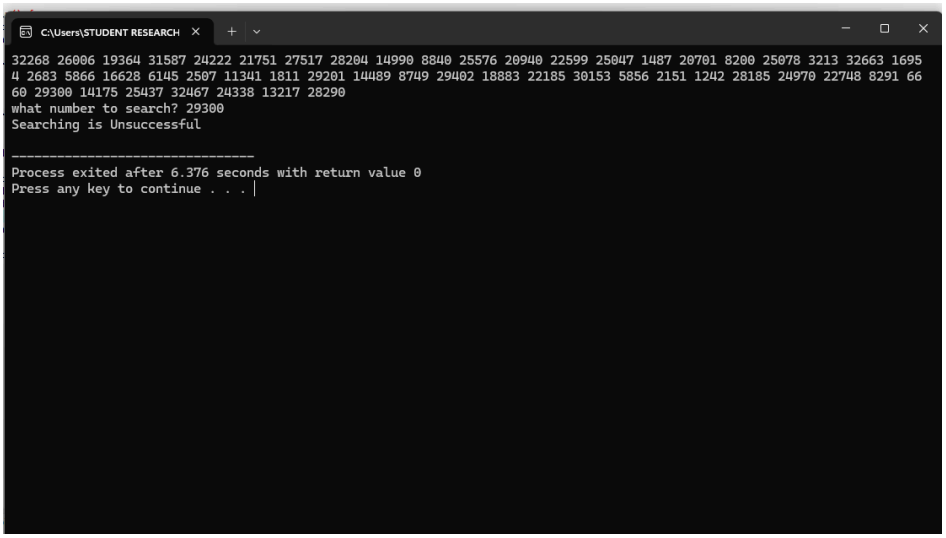
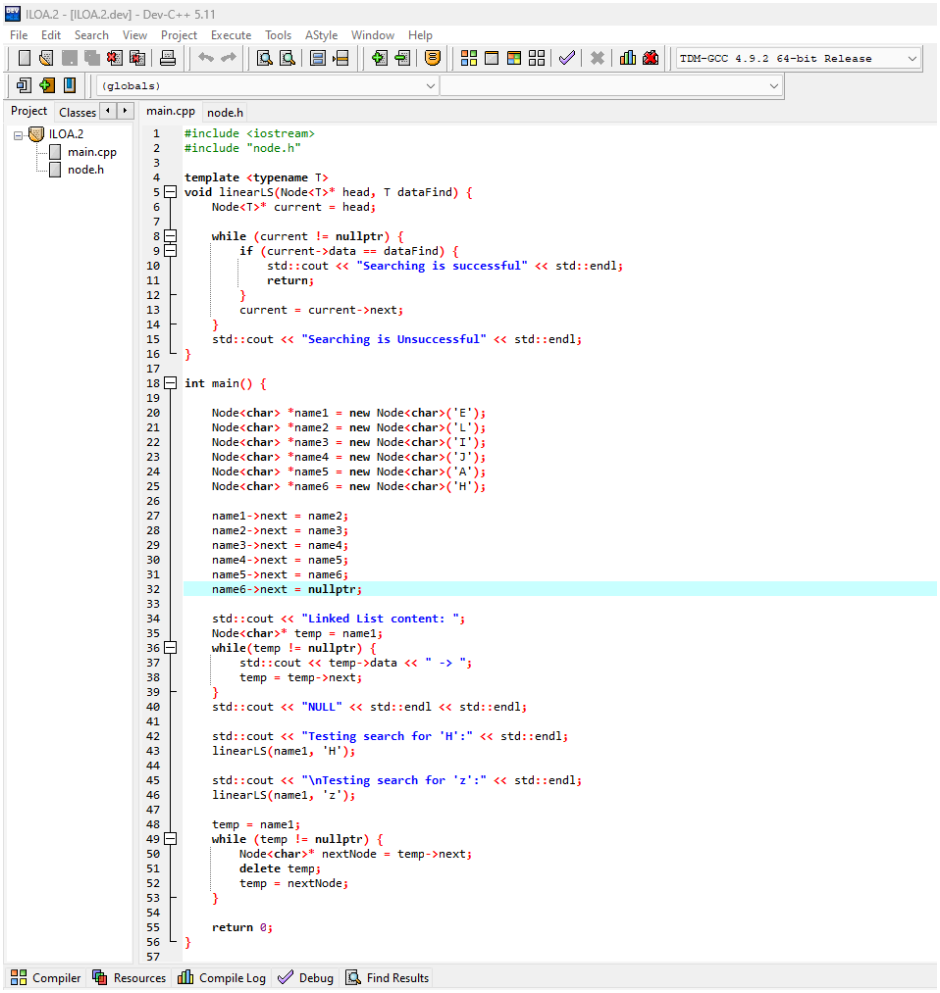
	
Observation	Comparing the code from the previous one, whenever the user inputs 4-5 digits that are on the list it prints out a message 'searching is successful', but for sequential search, most 4-5 digits are inputted prints out a message 'searching is unsuccessful'. It will only print out a message 'searching is successful', if the target number randomly matches the value generated at the very first position of the array which is '31759' .

Table 6-2a. Linear Search for Arrays

Code


ILOA.2 - [ILOA.2.dev] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release
(globals)
Project Classes
ILOA.2
main.cpp
node.h
main.cpp node.h
#include <iostream>
#include "node.h"
template <typename T>
void linearLS(Node<T>* head, T dataFind) {
Node<T>* current = head;
while (current != nullptr) {
if (current->data == dataFind) {
std::cout << "Searching is successful" << std::endl;
return;
}
current = current->next;
}
std::cout << "Searching is Unsuccessful" << std::endl;
}
int main() {
Node<char> *name1 = new Node<char>('E');
Node<char> *name2 = new Node<char>('L');
Node<char> *name3 = new Node<char>('I');
Node<char> *name4 = new Node<char>('J');
Node<char> *name5 = new Node<char>('A');
Node<char> *name6 = new Node<char>('H');
name1->next = name2;
name2->next = name3;
name3->next = name4;
name4->next = name5;
name5->next = name6;
name6->next = nullptr;
std::cout << "Linked List content: ";
Node<char>* temp = name1;
while(temp != nullptr) {
std::cout << temp->data << " -> ";
temp = temp->next;
}
std::cout << "NULL" << std::endl << std::endl;
std::cout << "Testing search for 'H':" << std::endl;
linearLS(name1, 'H');
std::cout << "\nTesting search for 'z':" << std::endl;
linearLS(name1, 'z');
temp = name1;
while (temp != nullptr) {
Node<char>* nextNode = temp->next;
delete temp;
temp = nextNode;
}
return 0;
}
Compiler Resources Compile Log Debug Find Results

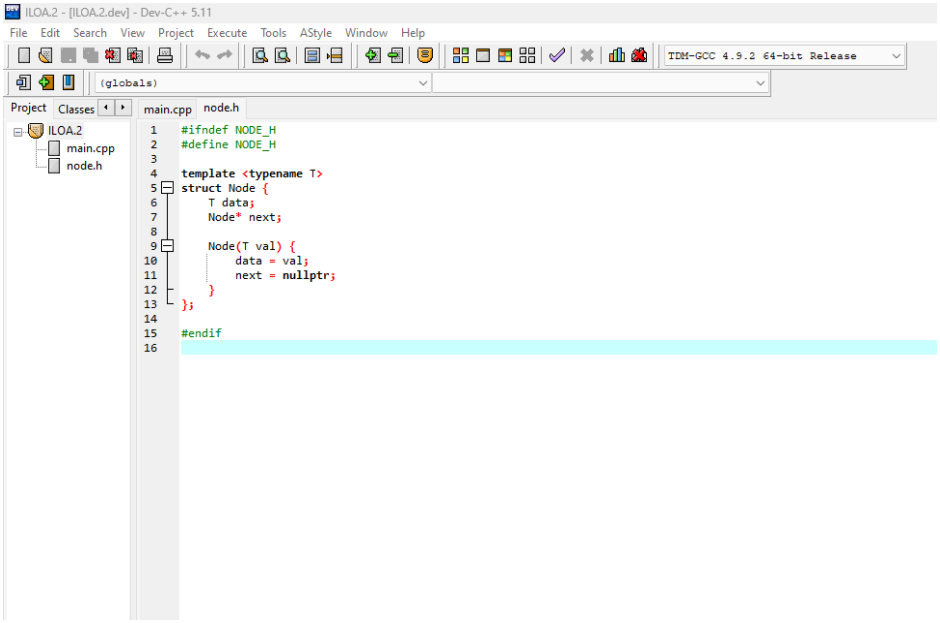
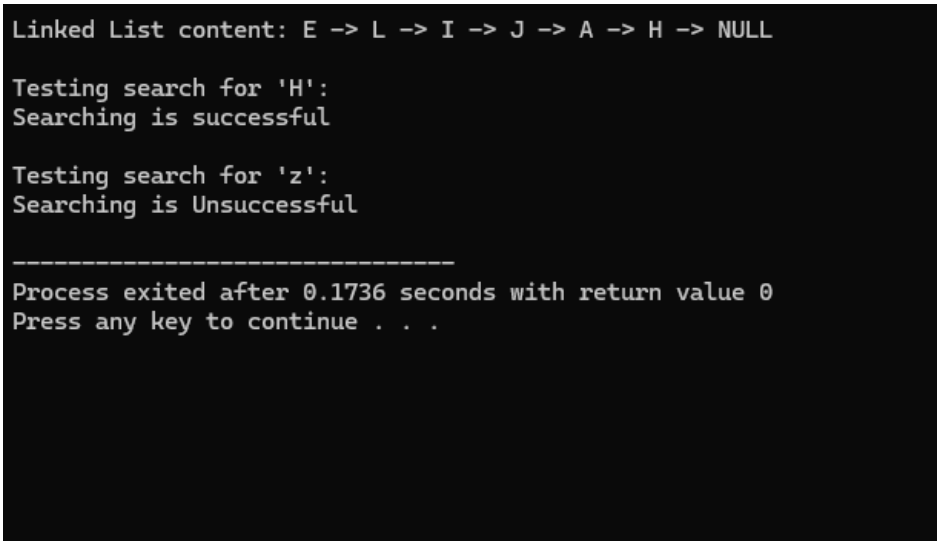
<table>
<tr><td></td><td> 
</td></tr>
<tr><td>Output</td><td> 
</td></tr>
<tr><td>Observation</td><td>It observes the start position of the search by initializing a temporary traversal pointer to point to the node of the list. It then creates a sequential scan using a while loop structure that ensures the execution automatically stops once the final node points out of bounds, avoiding memory access errors.</td></tr>
</table>

Table 6-2b. Linear Search for Linked List

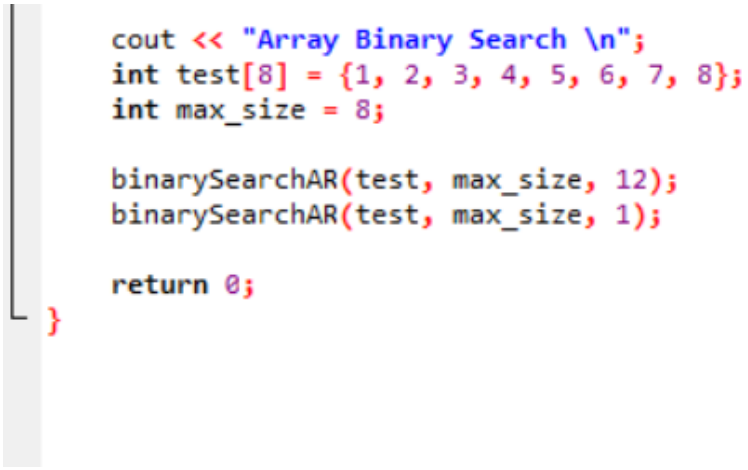
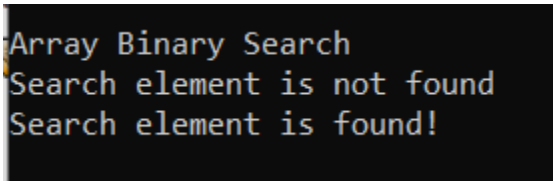
Code	
Output	
Observation	It sets up two pointers at the ends of the array, 'low = 0' that points to the first element, and 'up = 7, that points to the last element. The function then enters in a while loop that continues to run as long as the pointer of the first element has not crossed the last element.

Table 6-3a. Binary Search for Arrays

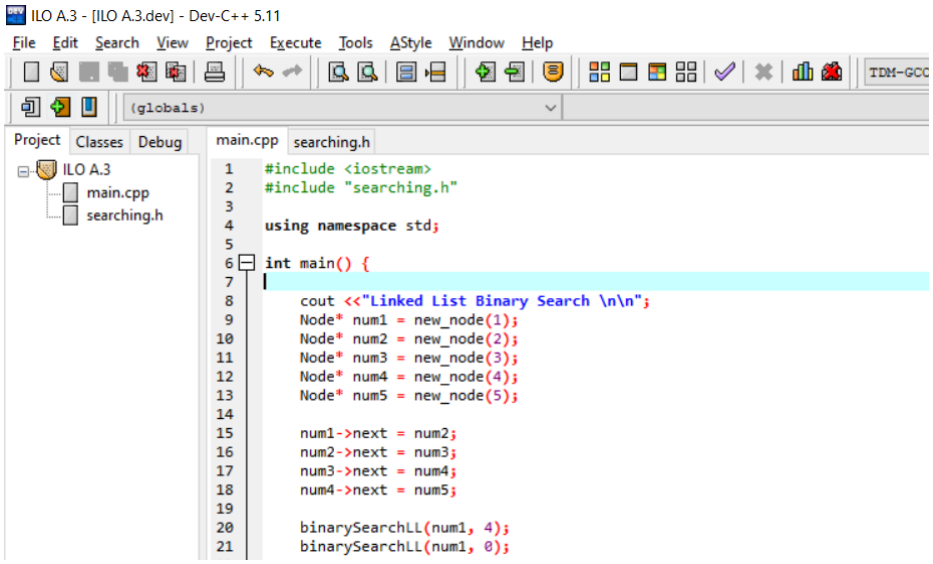
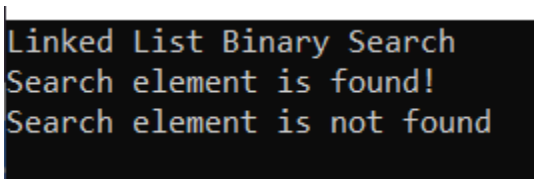
Code	
Output	
Observation	Based on the code, it creates a start and end pointer to track the active search area. Because the nodes do not have index numbers, the algorithm uses a slow and fast pointer to traverse the list and pinpoint the middle node.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C: Solve different problems utilizing appropriate searching techniques in C++

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons Would it be necessary to find the key '18'?

Code

\OneDrive\Desktop\HOA6.1\problem 1.cpp - Dev-C++ 5.11

```cpp
#include <iostream>
#include <vector>

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

void sequentialSearchArray(const std::vector<int>& arr, int target) {
    int comparisons = 0;
    bool found = false;
    int indexFound = -1;

    for (size_t i = 0; i < arr.size(); ++i) {
        comparisons++;
        if (arr[i] == target) {
            found = true;
            indexFound = i;
            break;
        }
    }

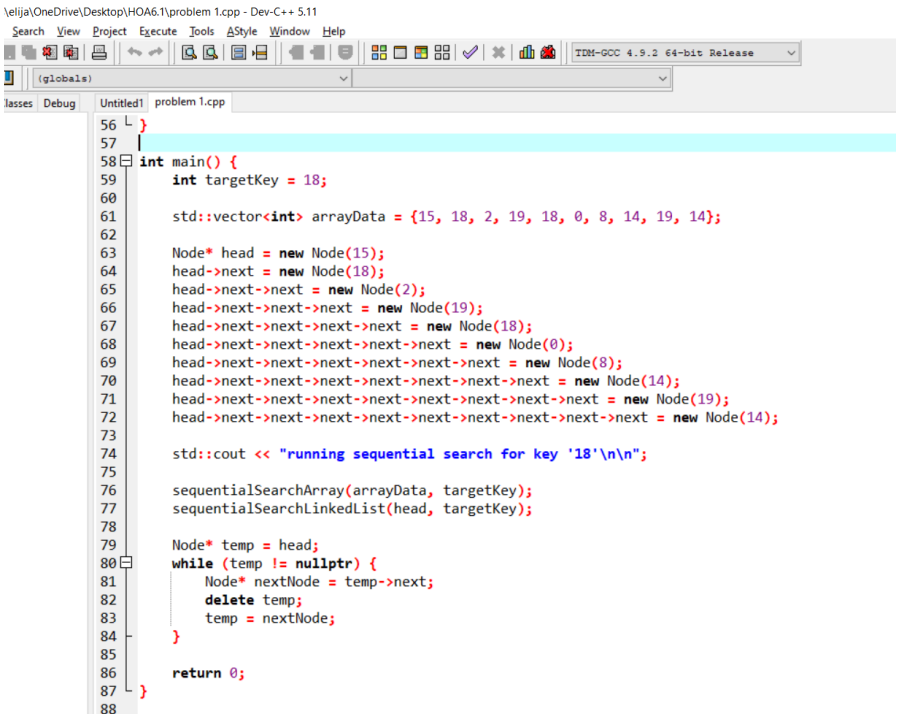
    std::cout << "target key '" << target << "' ";
    if (found) {
        std::cout << "found at index " << indexFound << ".\n";
    } else {
        std::cout << "not found.\n";
    }
    std::cout << "total comparisons: " << comparisons << "\n\n";
}
```

\elija\OneDrive\Desktop\HOA6.1\problem 1.cpp - Dev-C++ 5.11

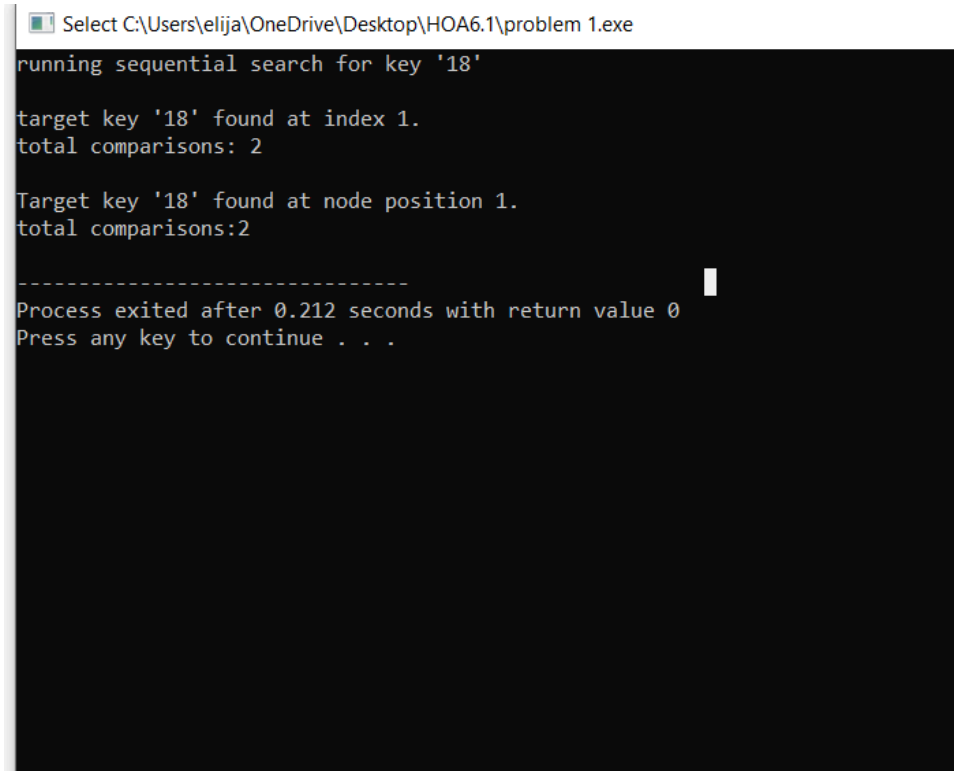
```cpp
void sequentialSearchLinkedList(Node* head, int target) {
    int comparisons = 0;
    bool found = false;
    Node* current = head;
    int position = 0;

    while (current != nullptr) {
        comparisons++;
        if (current->data == target) {
            found = true;
            break;
        }
        current = current->next;
        position++;
    }

    std::cout << "Target key '" << target << "' ";
    if (found) {
        std::cout << "found at node position " << position << ".\n";
    } else {
        std::cout << "not found.\n";
    }
    std::cout << "total comparisons:" << comparisons << "\n";
}
```



Output

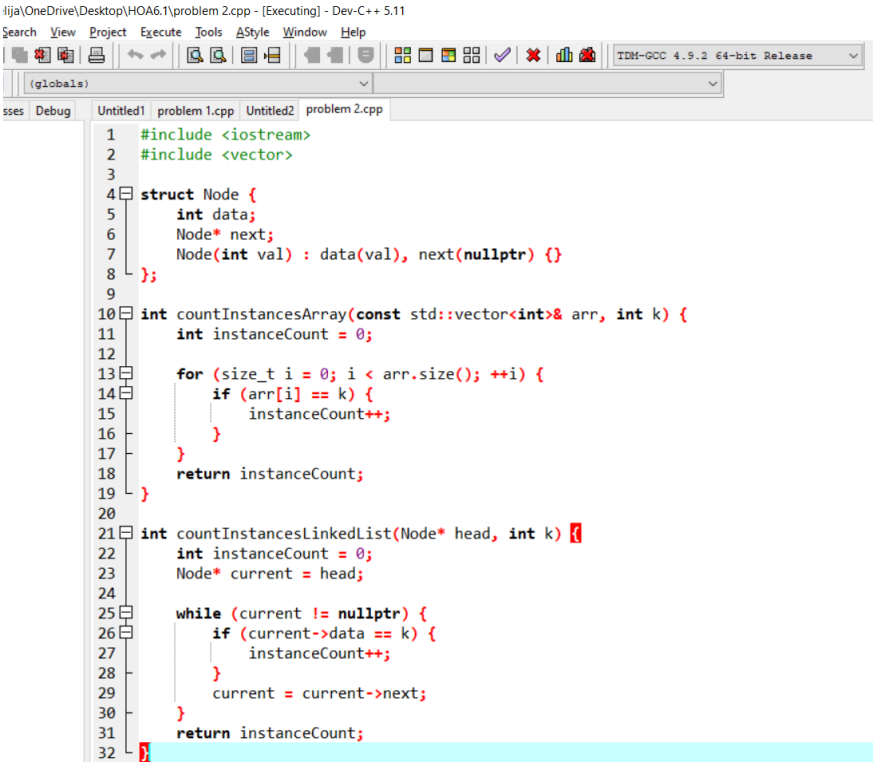


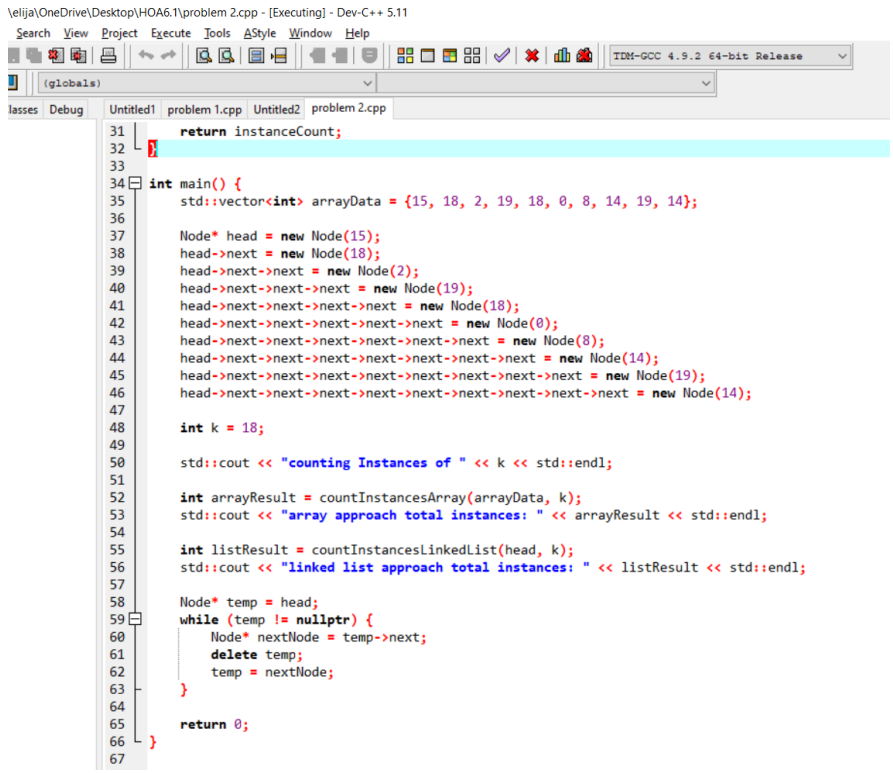
Observation:

The sequential search functions by checking every item one by one, starting from the first element and moving sequentially forward. The data set contains the target key twice, because the 'break' statement only finds and reports the first occurrence and ignores any duplicates later in the sequence.

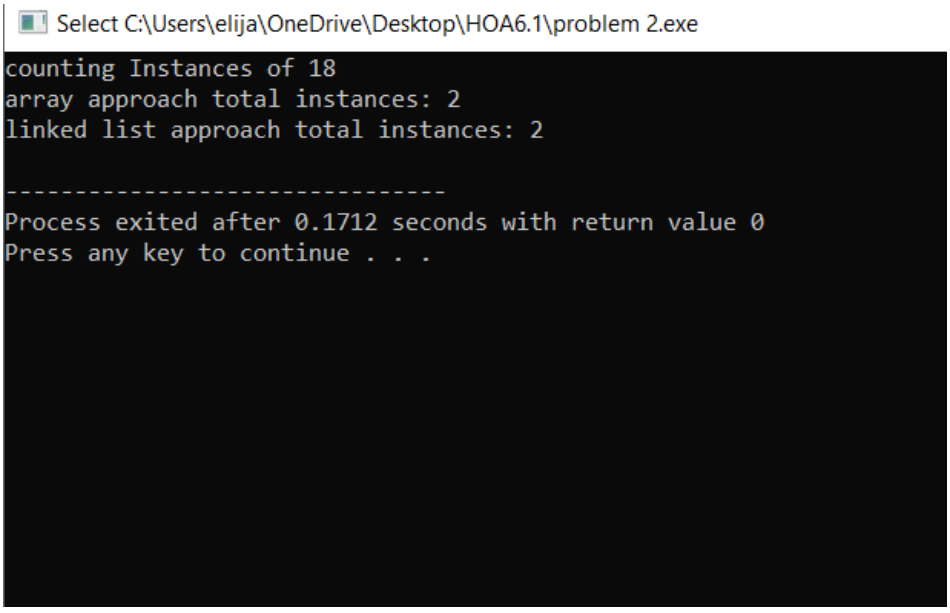
Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element 'k'. Test on the same list given in problem 1

Code	





Output



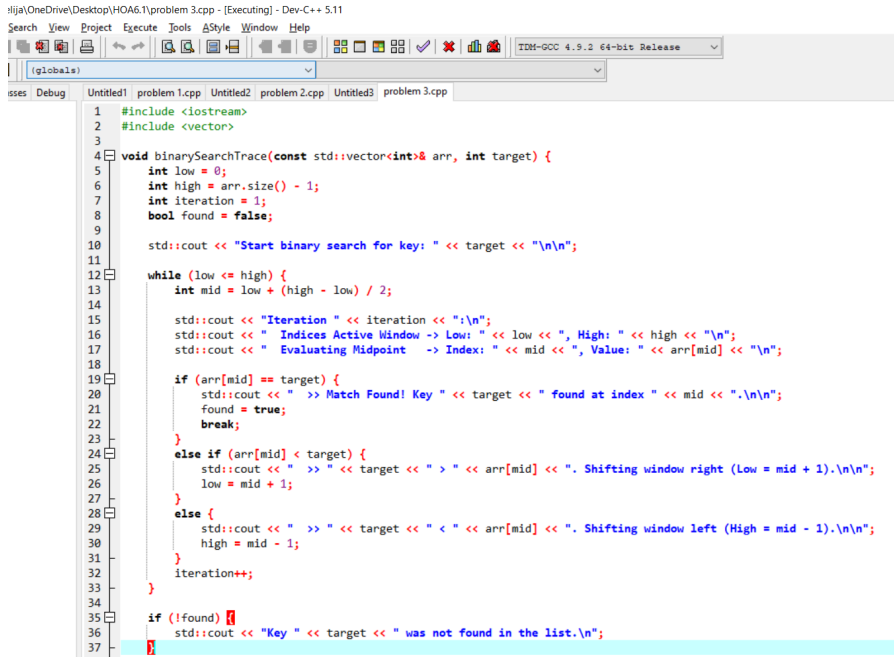
Observation:

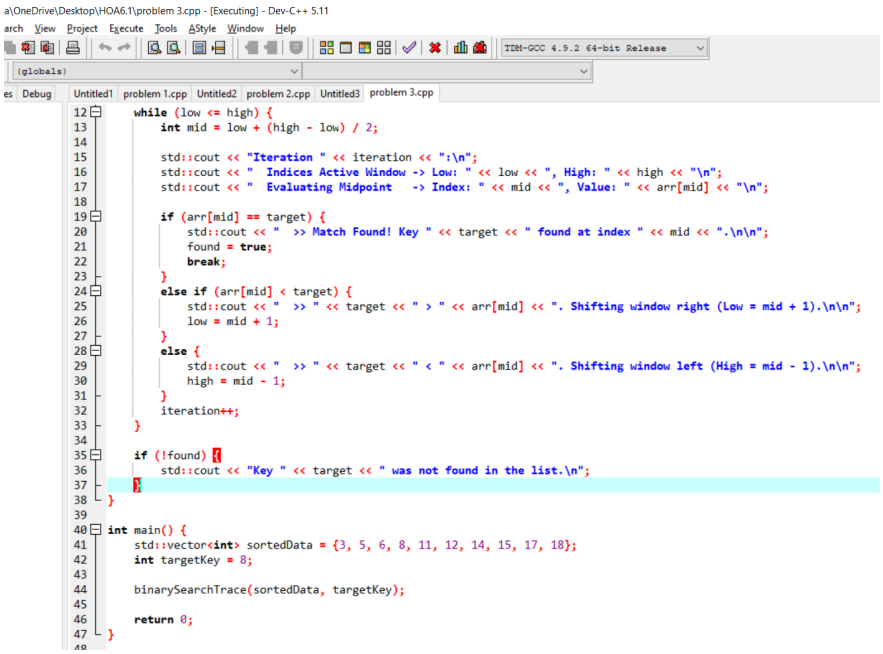
It utilizes a linear traversal path that follows pointer links from the head node down to a null pointer marker. It

inspects the internal integer value of each node against the target variable. It lacks an exit mechanism as it visits every node in the sequence to accurately count all duplicated values.

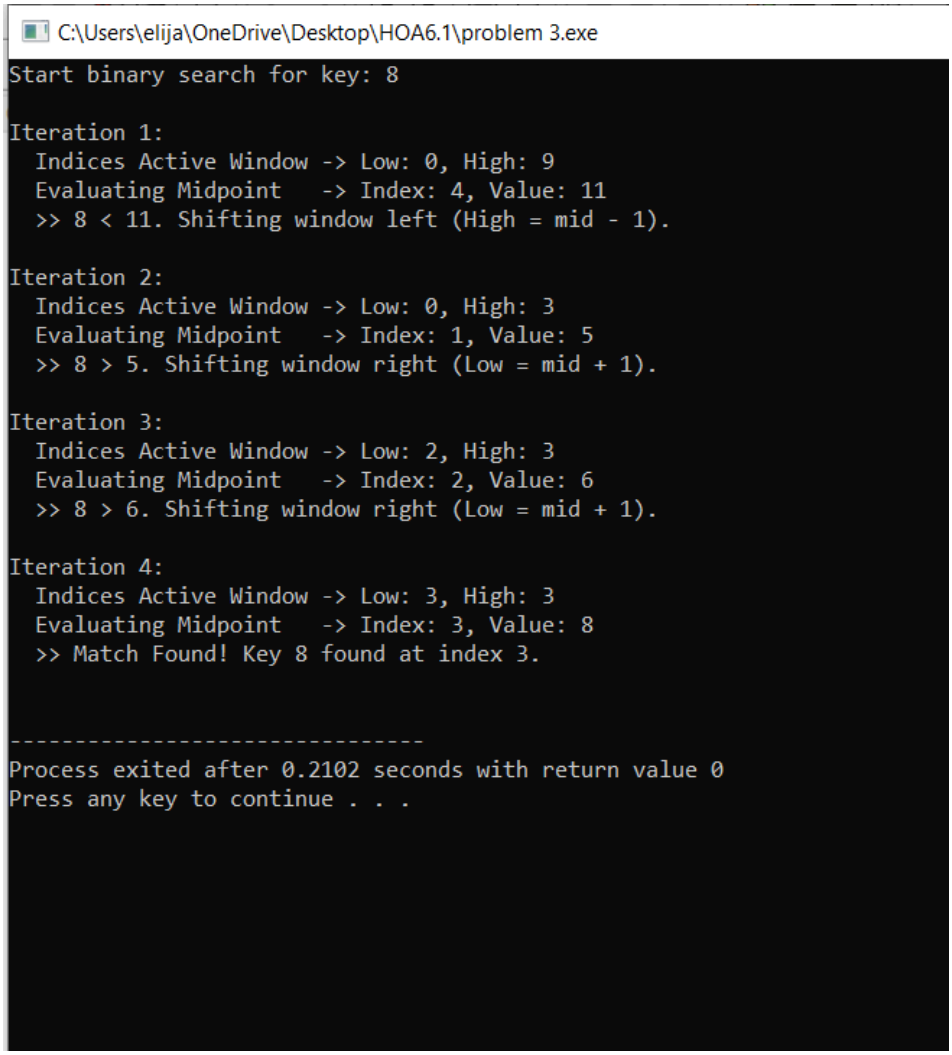
Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Code




a\OneDrive\Desktop\HOA6.1\problem 3.cpp - [Executing] - Dev-C++ 5.11
arch View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release
(globals)
es Debug
Untitled1 problem 1.cpp Untitled2 problem 2.cpp Untitled3 problem 3.cpp
12 while (low <= high) {
13 int mid = low + (high - low) / 2;
14
15 std::cout << "Iteration " << iteration << ":\n";
16 std::cout << " Indices Active Window -> Low: " << low << ", High: " << high << "\n";
17 std::cout << " Evaluating Midpoint -> Index: " << mid << ", Value: " << arr[mid] << "\n";
18
19 if (arr[mid] == target) {
20 std::cout << " >> Match Found! Key " << target << " found at index " << mid << ".\n\n";
21 found = true;
22 break;
23 }
24 else if (arr[mid] < target) {
25 std::cout << " >> " << target << " > " << arr[mid] << ". Shifting window right (Low = mid + 1).\n\n";
26 low = mid + 1;
27 }
28 else {
29 std::cout << " >> " << target << " < " << arr[mid] << ". Shifting window left (High = mid - 1).\n\n";
30 high = mid - 1;
31 }
32 iteration++;
33 }
34
35 if (!found) {
36 std::cout << "Key " << target << " was not found in the list.\n";
37 }
38 }
39
40 int main() {
41 std::vector<int> sortedData = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
42 int targetKey = 8;
43
44 binarySearchTrace(sortedData, targetKey);
45
46 return 0;
47 }

Output

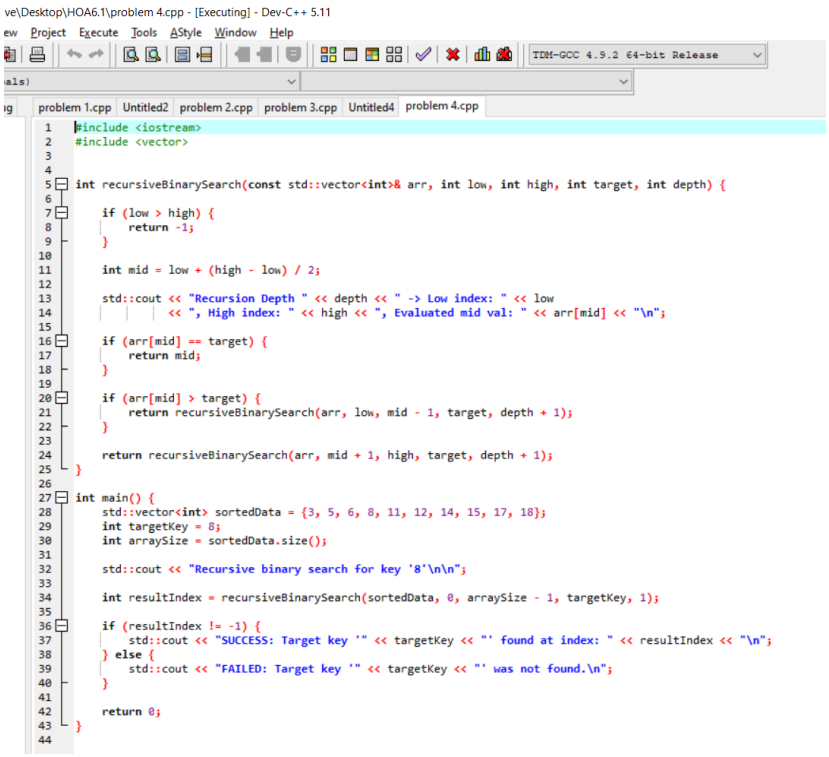


Observation:

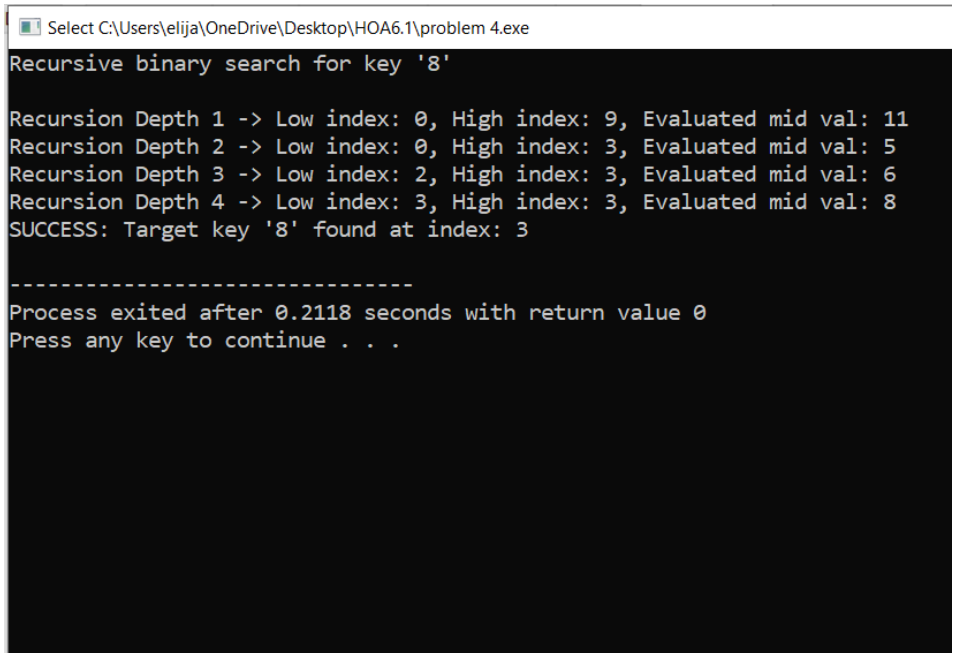
The code executes a tracing loop that outputs the internal state of the search boundaries at each step. It sets a lower and upper index range limit across a pre-sorted dataset that calculates the central middle position and prints out the active numbers, by evaluating whether the target value is small or larger than the midpoint value, it then shrinks the operation until a direct index match is found.

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3

Code



Output



Observation:

It passes the bounding ranges of ‘low’ and ‘high’ along with a tracking variable directly into the self-referencing function class, by evaluating the value at the calculated midpoint against the target value. The program discards the invalid half of the sorted collection and spawns a new and narrow recursion layer until a match is confirmed or the base condition breaks the chain.

Conclusion:

I learned that searching logic can be constructed using either iterative loop statements or recursive function stacks, while iterative approaches track state manually within a single execution block, recursive designs leverage continuous self-referencing calls to divide data problem spaces cleanly, showing that identical algorithmic logic can be achieved through different programming paradigms.

Based on how well I did in completing the activity, I would say good enough . But from my areas of improvement, there is a lot more such as trying to implement other searching algorithm techniques, being able to create a pseudocode and learning how to visualize them.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.’s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.